

7.3 BERT



#01 모델 설명

#1 BERT란?

- 2018년 구글에서 발표한 언어 모델
- 트랜스포머 기반 양방향 인코더 사용
- 대규모 데이터로 사전 학습 되어 있음 => 전이 학습에 주로 활용됨
- 일부/전체를 다른 작업에서 재사용 => 적은 양의 데이터로 높은 정확도 달성, 모델 학습 시간 단축 가능

#2 기존 모델과의 차이점

- 기존 모델: 문장을 좌 -> 우로 순차적으로 학습. 단어의 의미, 문맥 파악할 때 이전 단어만 고려
- 양방향(Bidirectional) 인코더: 입력 시퀀스를 양쪽 방향에서 처리.
이전과 이후 단어를 모두 참조하면서 의미, 문맥 파악해서 더 정확하고 다양한 자연어 처리 작업에서 높은 성능을 보임

#01 모델 설명

#3 BERT 구조

- 트랜스포머의 인코더 모듈만 사용해서 입력 문장을 처리함
- 인코더 계층만 사용해서 학습함
- 인코더: 입력 문장의 단어들을 임베딩해서 각 단어의 의미를 벡터화 -> 순차적으로 처리 -> 문장 전체의 의미 추출
- BERT: 입력 문장의 단어를 좌우 양방향으로 처리 -> 문맥 정보 모델링

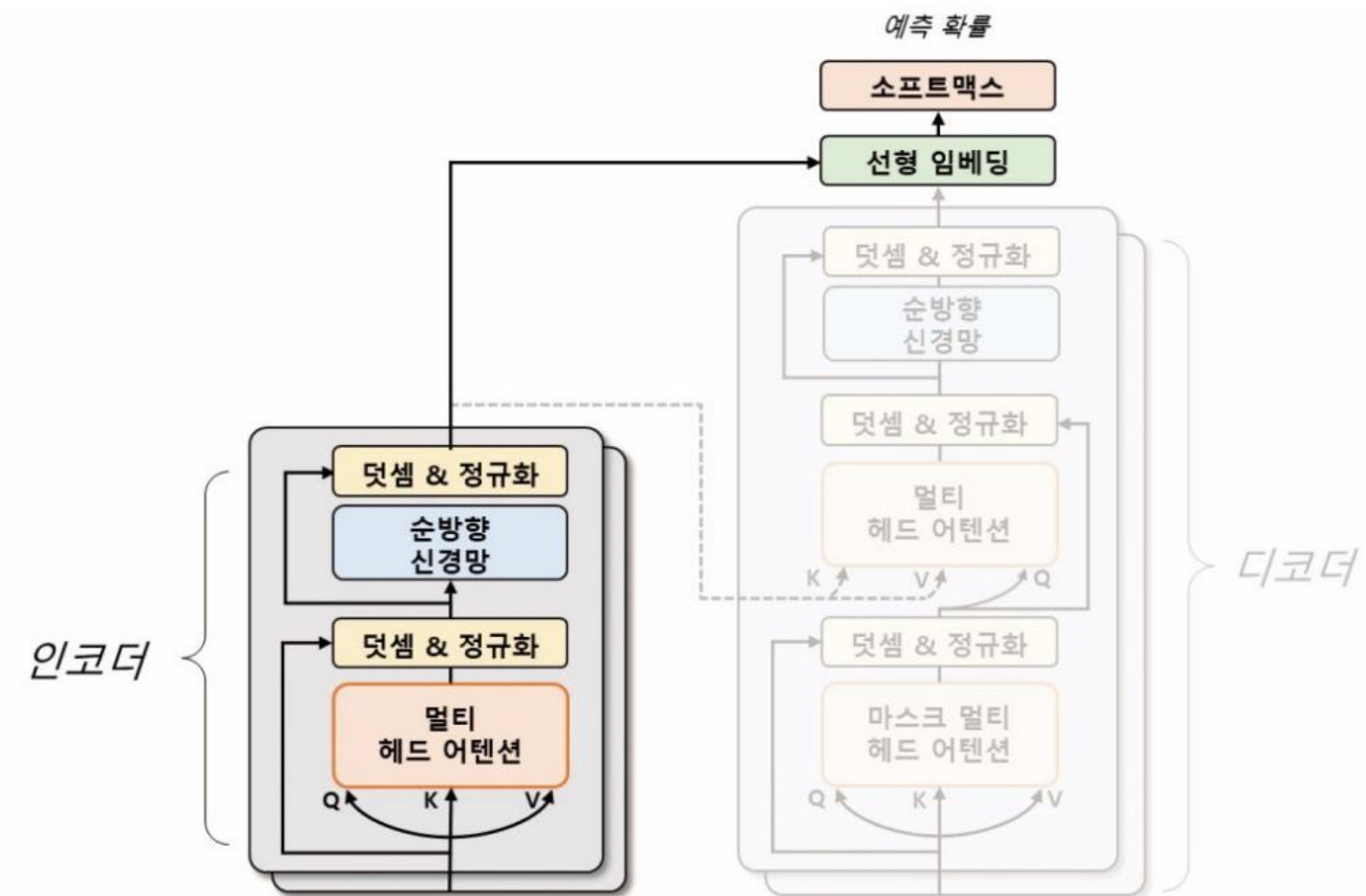


그림 7.13 트랜스포머와 BERT 모델 비교

#02 사전 학습 방법

#1 MLM (Masked Language Modeling)

- 마스크된 언어 모델
- 입력 문장에서 임의로 일부 단어를 마스크하고 해당 단어를 예측하는 방식
- 양방향으로 문장 학습해서 문맥 정보 모델링해서 입력 문장에서 누락된 단어 추론 가능
- 문장 전체의 의미를 이해하는 능력 향상
- 예시) I'm learning PyTorch
 - Learning 마스크 => I'm [MASK] PyTorch => BERT가 [MASK]에 들어갈 단어 예측 (양방향 문맥 정보 참고)

#2 NSP (Next Sentence Prediction)

#3 contents

#02 사전 학습 방법

#1 MLM (Masked Language Modeling)

- 마스크된 언어 모델
- 입력 문장에서 임의로 일부 단어를 마스크하고 해당 단어를 예측하는 방식
- 양방향으로 문장 학습해서 문맥 정보 모델링해서 입력 문장에서 누락된 단어 추론 가능
- 문장 전체의 의미를 이해하는 능력 향상
- 예시) I'm learning PyTorch
 - Learning 마스크 => I'm [MASK] PyTorch => BERT가 [MASK]에 들어갈 단어 예측 (양방향 문맥 정보 참고)

#2 NSP (Next Sentence Prediction)

- 다음 문장 예측
- 두개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장 다음에 오는 문장인지 판단하는 작업
- 예시) I'm learning PyTorch, PyTorch is a machine learning library.
 - BERT가 이 두 문장이 연속적인 관계인지 아닌지 예측함
 - BERT가 두 문장 간의 관계를 학습, 문장 간의 의미적인 유사성 파악함

#02 사전 학습 방법

#4 토큰 - [CLS]

- 입력 문장의 시작 부분에 추가되는 토큰
- 문장 분류 작업을 위한 정보 제공
- 모델이 입력 문장이 어떤 유형인지 미리 파악할 수 있게 함
 - 긍정/부정적
 - 문장 내에서 어떤 객체가 언급되는가

#5 토큰 - [SEP]

- 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용되는 토큰
- 문장 분류 작업 단계에서 두 개 문장 받을 때 [SEP] 토큰으로 두 문장 구분함
- 입력 문장

#6 토큰 - [MASK]

- 입력 문장 내에서 임의로 선택된 단어를 가리키는 토큰
- 문장에서 단어 일부 가리고 모델 학습, 예측에 사용함
모델링 작업에서 실제 단어 예측하는데 사용함

#02 사전 학습 방법

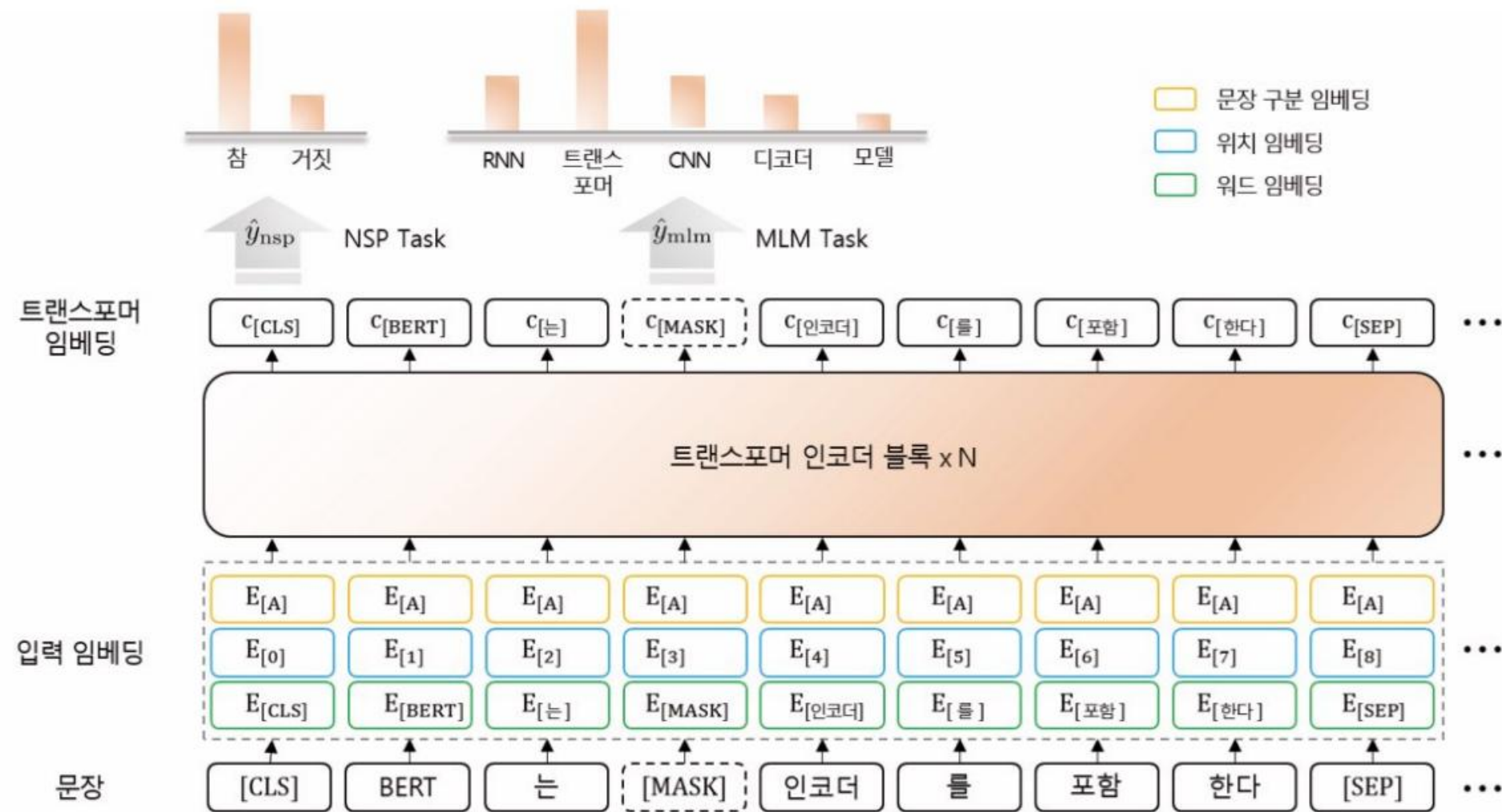


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

- 입력 문장: BERT는 트랜스포머 인코더를 포함한다
- [MASK]: 텍스트 토큰 중 15%에 마스킹 (80%: [MASK], 10%: 사전에 존재하는 무작위 단어, 10%: 실제 토큰 그대로)
- [CLS]: 분류
- 사전 학습 수행 -> 미세 조정 기법 -> 자연어 처리 작업
- 미세 조정: 해당 작업에 맞는 계층 추가, 손실 함수 정의해서 학습
- 자연어 처리 작업: 문장 분류, 감성 분석, 질문 응답, 기계 번역

#03 모델 실습

#1 네이버 영화 리뷰

```
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
df = pd.DataFrame(corpus.test).sample(20000, random_state=42)
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)
```

```
print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

데이터셋 크기가 작은 테스트 데이터셋을 활용해 학습을 진행함

- 60%: 학습 데이터
- 20%: 검증 데이터
- 20%: 테스트 데이터

#03 모델 실습

#2 BERT 입력 텐서

- BERT 토크나이저 클래스로 데이터 전처리하고 데이터로더에 적용
- 사전 학습된 모델: bert=base-multilingual-cased
 - 다중 언어 지원
 - 대소문자 유지
 - 소문자 유지(do_lower_case) False => 소문자로 변환하지 않게 함
 - 소문자 유지 True => Apple, apple 구분
 - 모든 언어 데이터 한번에 학습해서 다국어 자연어 처리 작업에 적합
- 토크나이저를 make_dataset 함수에 전달해서 텐서 데이터셋 반환
 - 텍스트 -> 숫자ID, 어텐션 마스크
 - 레이블 -> 텐서
- 데이터로더 적용
 - get_dataloader: 샘플러 클래스를 활용해서 데이터 목적에 따라 샘플링함
 - RandomSampler: 데이터를 무작위로 샘플링 -> 학습에 적용
 - SequentialSampler: 데이터를 고정된 순서대로 리턴 -> 검증, 평가 배치

```
import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"
tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    do_lower_case=False
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])
```

#03 모델 실습

#3 BERT 모델 선언

예제 7.17 BERT 모델 선언

```
from torch import optim
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- BertForSequenceClassification: BERT 문장 분류 모델
- AdamW: Adam 최적화 함수에 가중치 감소를 추가한 변형된 경사 하강법 알고리즘
 - eps: 학습률을 0으로 나누는 것을 방지하기 위해 분모에 더함
 - 갱신되는 모든 매개변수의 학습률을 조절해서 모델이 빠르게 수렴하고 안정적인 학습 가능

#03 모델 실습

#4 BERT 모델 구조

bert	- 임베딩
└ embeddings	- 토큰 임베딩 (word_embeddings)
└ word_embeddings	- 위치 임베딩 (position_embeddings)
└ position_embeddings	- 문장 구분 임베딩 (token_type_embeddings)
└ token_type_embeddings	- 서로 다른 정보를 담고있는 임베딩들은 결합해서 입력 시퀀스에 대한 임베딩 벡터를 생성함
└ LayerNorm	- 이후 생성된 임베딩 벡터: 정규화, 드롭아웃
└ dropout	
└ encoder	- 인코더 블록
└ layer	- 블록 개수: 12개
└ 0	- 입력 시퀀스를 순차적으로 처리 -> 새로운 임베딩 생성
└ 1	- 셀프 어텐션, 순방향 신경망 사용 -> 입력 시퀀스의 다양한 관계 학습
...	
└ 9	
└ 10	
└ 11	
└ pooler	- 풀러(pooler): [CLS] 토큰 벡터를 한 번 더 비선형 변환 과정을 수행하기 위해 Tanh 함수 사용
└ dense	- 드롭 아웃 적용: overfitting 방지
└ activation	- 분류기(classifier): [CLS] 토큰 벡터 활용해서 결과 예측
dropout	
classifier	

#03 모델 실습

#5 모델 평가 결과

BERT 모델 학습 결과

```
Epoch 1: Train Loss: 0.5744 Val Loss: 0.4519 Val Accuracy 0.7867
Saved the model weights
Epoch 2: Train Loss: 0.4224 Val Loss: 0.4503 Val Accuracy 0.7957
Saved the model weights
Epoch 3: Train Loss: 0.3320 Val Loss: 0.4207 Val Accuracy 0.8147
Saved the model weights
```

BERT 모델 평가 결과

```
Test Loss : 0.4183
Test Accuracy : 0.8085
```

- 학습을 반복할수록
 - 학습 손실 감소
 - 검증 손실, 검증 정확도 점진적으로 개선됨
- Overfitting 되지않고, 적절하게 학습되고 있음!
- 모델이 새로운 데이터에 대해 어느정도 일반화 된 성능을 보이고 있음

BART



BART

BART: 2019년 메타에서 발표한 트랜스포머
기반 모델

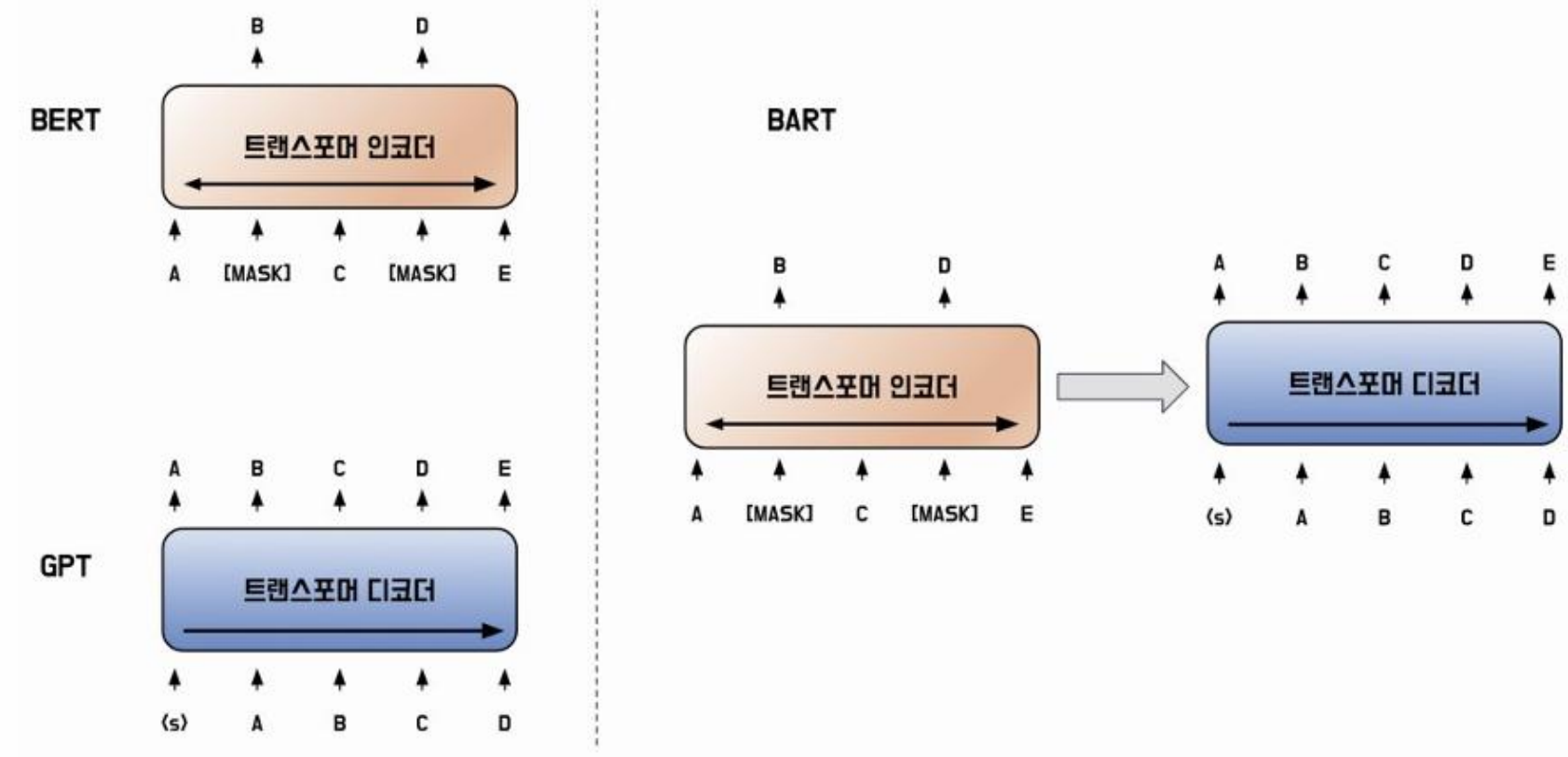
- BERT의 인코더와 GPT의 디코더를 결합한 시퀀스-시퀀스 구조

- 노이즈 제거 오토인코더로 사전 학습됨 -> 입력 데이터에 잡음 추가, 잡음이 없는 원본 데이터를 복원하도록 학습하는

방식으로 수행

즉, 노이즈가 추가된 텍스트를 인코더에 입력 -> 원본 텍스트를 디코더에 입력해 디코더가 원본 텍스트를 생성하는 방식

BART



BERT의 인코더

- 입력 문장에서 일부 단어를 무작위로 마스킹해 처리하고, 마스킹된 단어를 맞추도록 학습
- 문장 전체 맥락을 이해하고 문맥 내 단어 간 상호작용 파악 가능

GPT

- 언어 모델을 이용해 문장의 이전 토큰들을 입력으로 받고, 다음에 올 토큰을 맞추도록 학습
- 문장 내 단어들의 순서와 맥락을 파악하며, 다음에 올 단어를 예측하는 능력 가짐

BART

이 구조를 통해 BART는

- 문장 구조와 의미를 보존하면서 다양한 변형 학습 가능
- 더 풍부한 언어적 지식 습득 가능

인코더를 사용함으로써 순방향 정보만 인식할 수 있는 GPT 단점 개선, 양방향 문맥 정보 반영
디코더를 사용함으로써 문장 생성 분야에서 뛰어나지 않았던 BERT의 단점을 개선

BERT와 BART의 차이점

- BERT는 인코더만 사용
- BART는 인코더와 디코더 모두 사용. 문장 생성, 번역 작업에 적합. 문장 독해, 문서 요약, 질의응답 등에 있어 높은 성능을 보임

BART

BART와 트랜스포머의 차이점

- 트랜스포머: 인코더의 모든 계층과 디코더의 모든 계층 사이의 어텐션 연산을 수행
- BART: 인코더의 마지막 계층과 디코더의 각 계층 사이에만 어텐션 연산을 수행 -> 정보 전달 최적화하고 메모리 사용량 줄일 수 있음

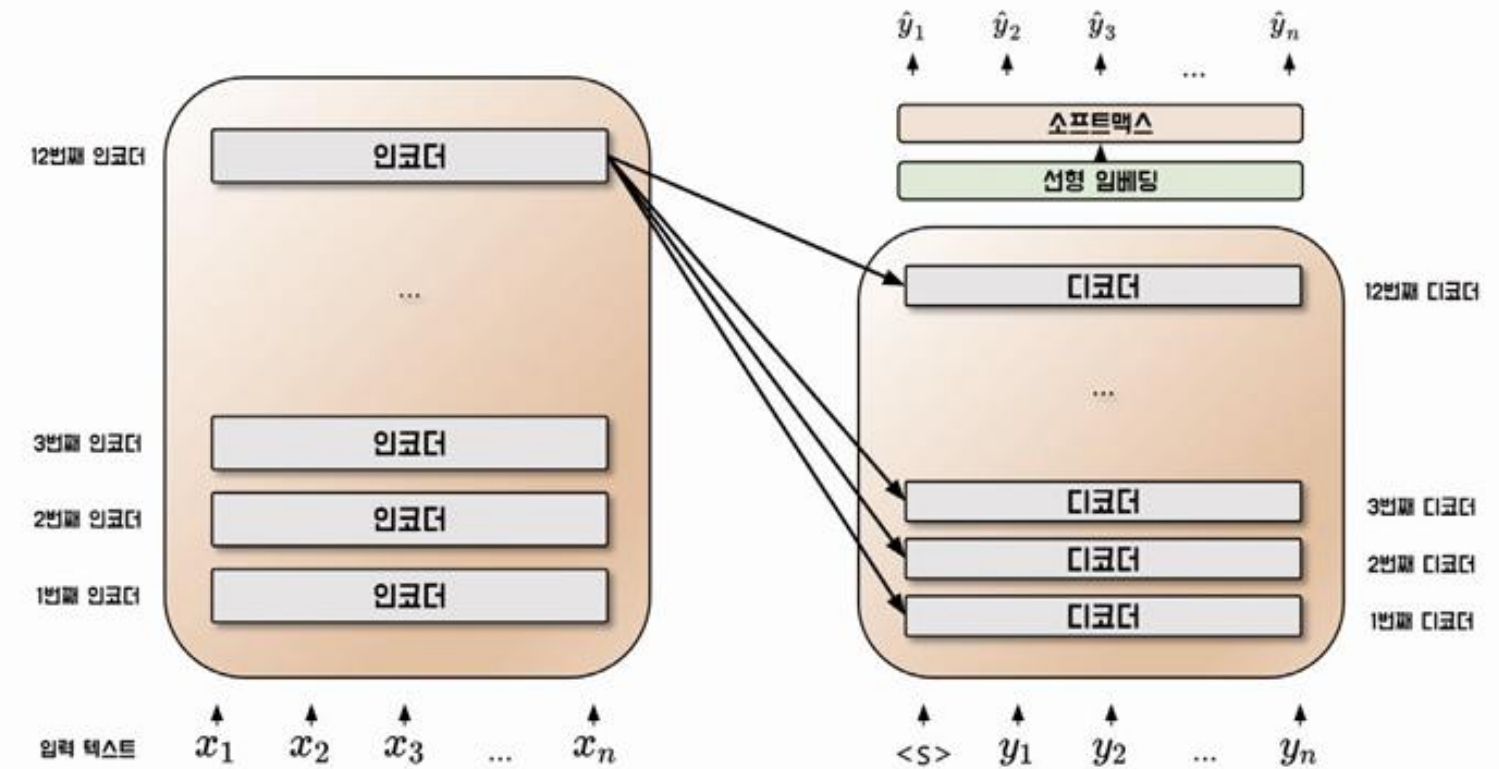


그림 7.16 BART 모델 구조

사전 학습 방법

BART 인코더는 BERT의 마스킹된 언어 모델링(MLM) 이외에도 다양한 노이즈 기법 사용(토큰 마스킹 기법, 토큰 삭제, 문장 교환, 문서 회전, 텍스트 채우기 기법 등)

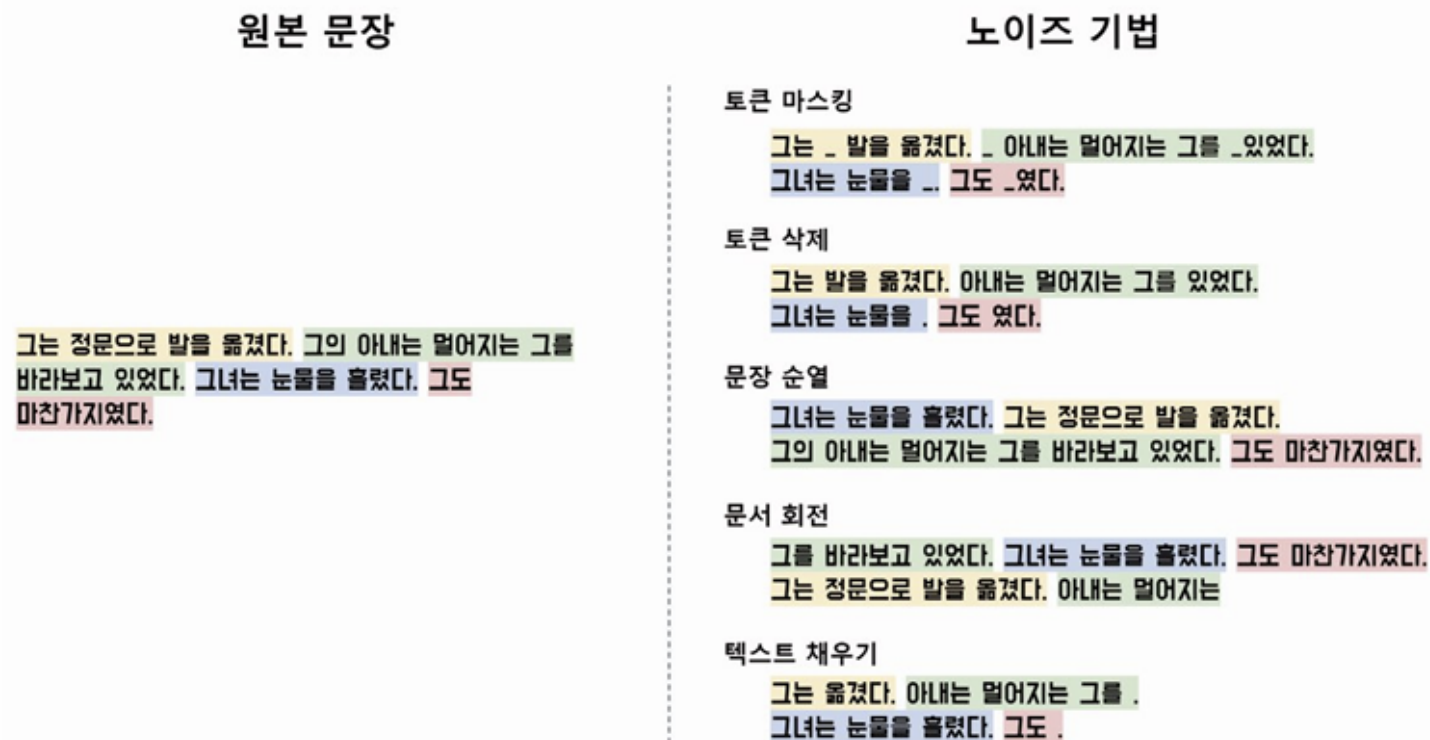


그림 7.17 BART 노이즈 기법

토큰 마스킹: 입력 문장의 일부 토큰을 마스크 토큰으로 치환하는 방법

토큰 삭제: 입력 문장의 일부 토큰을 삭제하는 방법

문장 순열: 마침표(.)를 기준으로 문장을 나눈 뒤, 문장의 순서를 섞는

방범

문서 회전: 임의의 토큰으로 문서가 시작하도록 함

텍스트 채우기: 몇 개의 토큰을 하나의 구간(span)으로 묶고

일부 구간을 마스크 토큰으로 대체

미세 조정 방법

BART는 인코더와 디코더를 모두 사용하기 때문에 인코더와 디코더에 다른 문장 구조로

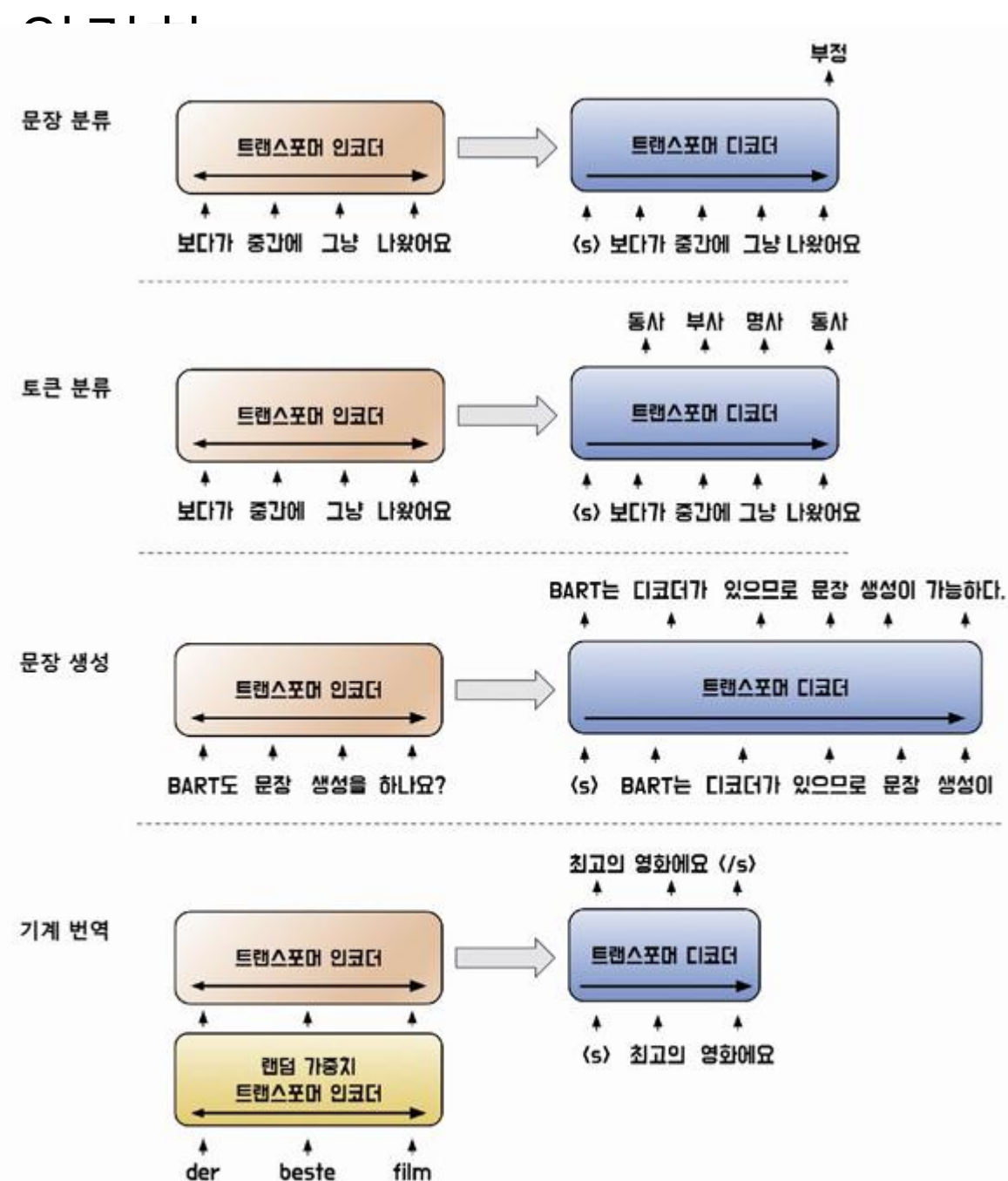


그림 7.18 BART 미세 조정 구조

문장 분류 작업: 입력 문장을 인코더와 디코더에 동일하게 입력하고 디코더 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용

토큰 분류 작업: 입력 문장을 인코더와 디코더에 동일하게 입력하고 디코더 각 시점별 마지막 토큰 은닉 상태를 토큰 분류기의 입력값으로 사용

디코더

디코더

미세 조정 방법

BART는 트랜스포머 디코더를 사용하기 때문에 BERT가 해결하지 못했던 문장 생성 작업을 수행할 수 있음

특히 입력값을 조작해 출력을 생성하는 추상적 질의응답과 문장 요약 작업에 적합
위와 같은 작업들이 BART의 사전 학습 방식과 유사하기 때문

학습은 두 단계로 이루어짐

1. 새로 추가된 트랜스포머 인코더의 가중치와 위치 임베딩, 첫 번째 인코더 층의 셀프 어텐션
입력 행렬 가중치만 학습
2. 모든 신경망의 가중치를 작은 반복으로 학습

모델 실습

- 허깅 페이스 라이브러리의 BART 모델
- 뉴스 요약 데이터셋(뉴스 본문 + 본문의 요약 텍스트)

➔ 문장 요약 모델 미세 조정

데이터셋은 허깅 페이스에서 제공하는 datasets 라이브러리를 통해 불러옴

Datasets 라이브러리 설치하기:

```
pip install datasets
```

모델 실습

뉴스 요약 데이터셋

불러오기

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])
train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

```
Source News : DANANG, Vietnam (Reuters) - Russian President Vladimir
Summarization : Putin says had useful interaction with Trump at Vi
Training Data Size : 3000
Validation Data Size : 1000
Testing Data Size : 1000
/usr/local/lib/python3.12/dist-packages/numpy/_core/fromnumeric.py:54
    return bound(*args, **kwds)
```

모델은 뉴스 본문을 입력으로 받아
요약된 텍스트를 출력해야 함

모델 실습

BART 입력 텐서 생성

```
def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt",
        max_length=1024
    )
    labels = []
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    for target in data.prediction:
        labels.append(tokenizer.encode(target, return_tensors="pt").squeeze())
    labels = pad_sequence(labels, batch_first=True, padding_value=-100).to(device)
    return TensorDataset(input_ids, attention_mask, labels)
```

- 테스트 데이터세트는 5,000개 샘플링해 사용
- 샘플링한 데이터세트 학습:검증:테스트 = 6:2:2
- 분리된 데이터세트를 BART 토크나이저(BartTokenizer) 클래스로 모델 구조에 적합하게 전처리 수행
- BART 토크나이저는 사전 학습된 토크나이저 불러와 전처리 수행
- 사전 학습된 모델은 facebook/bart-base로 BART 모델의 기본 버전 의미
- 문장 길이가 각각 다르기 때문에 길이 맞추기 위해 패딩 적용

모델 실습

BART 입력 텐서 생성

```
def get_dataloader(dataset, sampler, batch_size):  
    data_sampler = sampler(dataset)  
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)  
    return dataloader
```

데이터로더는 요약하려는 본문 텍스트의 정수 인코딩 값, 어텐션 마스크, 요약 문장의 정수 인코딩 값을 반환함

모델 실습

BART 모델 선언

```
from torch import optim
from transformers import BartForConditionalGeneration

model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
).to(device)
optimizer = optim.AdamW(model.parameters(), lr=5e-5, eps=1e-8)
```

- BartForConditionalGeneration 클래스는 BART 모델 변형 중 하나
- 빠른 학습을 위해 6개 인코더/디코더 계층 사용하는 facebook/bart-base 모델 사용
- 최적화 함수는 AdamW, 학습률은 0.00005

모델 실습

BART 모델 구조

```
model
├─ shared
├─ encoder
│  └─ embed_tokens
│  └─ embed_positions
│  └─ layers
│     └─ 0
│     └─ 1
│     └─ 2
```

```
│     └─ 3
│     └─ 4
│     └─ 5
└─ layernorm_embedding
├─ decoder
│  └─ embed_tokens
│  └─ embed_positions
│  └─ layers
│     └─ 0
│     └─ 1
│     └─ 2
│     └─ 3
│     └─ 4
│     └─ 5
└─ layernorm_embedding
lm_head
```

- BART는 인코더와 디코더가 동일한 임베딩 계층 사용
- Shared 계층은 인코더와 디코더가 공유하는 임베딩 계층 의미하며, 이러한 공유로 인코더와 디코더 간의 연결 강화
- 6개의 인코더와 디코더 계층으로 구성
- 마지막 연산에서는 layernorm_embedding 계층 통과
- layernorm_embedding 계층: 인코더와 디코더에서 각 토큰의 임베딩에 적용되는 계층 정규화로 임베딩 벡터의 마지막 차원에 대해 정규화를 수행해 학습 안정화
- 인코더 마지막 계층의 출력값은 디코더의 모든 계층과 어텐션 연산 수행
- 디코더 마지막 계층의 출력값은 출력 크기가 단어 사전 크기인 완전 연결

모델 실습

모델 평가 방법

- 문장 생성 기법에서 자주 사용되는 루지(ROUGE) 점수 사용
- 루지 점수: 생성된 요약문과 정답 요약문이 얼마나 유사한지를 평가하기 위해 토큰의 N-gram

정밀도와 재현율을 이용해 평가하는 지표

정답 문장 유니그램	[대한민국, 은, 16, 강, 에, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$
정답 문장 바이그램	[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$

그림 7.19 루지 점수 계산 방법

- ROUGE-L: 생성된 요약문과 정답 요약문 사이에서 최장 공통부분 수열(LCS) 찾는 통계 방식
- LCS: 두 문장 사이에 공통으로 존재하는 가장 긴 부분 문자열
- ROUGE-LSUM: 텍스트 내의 개행 문자를 문장 경계로 인식하고, 각 문장 쌍에 대해 LCS를 계산한 후, union-

모델 실습

- 허깅 페이스의 evaluate 라이브러리로 루지 점수 계산 가능

허깅 페이스 모델 평가 라이브러리

```
pip install evaluate rouge_score absl-py
```

모델 실습

BART 모델 학습 및 평가

```
import numpy as np
import evaluate

def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    rouge2 = rouge_score.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )
    return rouge2["rouge2"]

...
```

```
def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_rouge = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            outputs = model(
                input_ids=input_ids, attention_mask=attention_mask, labels=labels
            )
            logits = outputs.logits
            loss = outputs.loss

            logits = logits.detach().cpu().numpy()
            label_ids = labels.to("cpu").numpy()
            rouge = calc_rouge(logits, label_ids)

            val_loss += loss
            val_rouge += rouge

        val_loss = val_loss / len(dataloader)
        val_rouge = val_rouge / len(dataloader)
        return val_loss, val_rouge

rouge_score = evaluate.load("rouge", tokenizer=tokenizer)
```

- calc_rouge: 텍스트 요약
작업에서 모델이 예측한
요약문과 정답 요약문
사이의 루지 점수
계산하는 함수
- Preds: 모델이 예측한
요약의 토큰 인덱스를
담은 2차원 배열로,
argmax 메서드를 통해 각
토큰에서 가장 높은

```
... Downloading builder script: 6.14k/? [00:00<00:00, 151kB/s]
Epoch 1: Train Loss: 2.1932 Val Loss: 1.8763 Val Rouge 0.2359
Saved the model weights
Epoch 2: Train Loss: 1.6207 Val Loss: 1.8891 Val Rouge 0.2520
Epoch 3: Train Loss: 1.2527 Val Loss: 1.9453 Val Rouge 0.2268
Epoch 4: Train Loss: 0.9685 Val Loss: 2.0856 Val Rouge 0.2469
Epoch 5: Train Loss: 0.7413 Val Loss: 2.2326 Val Rouge 0.2322
```


모델 실습

BART 모델 학습 및 평가

```
import numpy as np
import evaluate

def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    rouge2 = rouge_score.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )
    return rouge2["rouge2"]

...
```

```
def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_rouge = 0.0, 0.0

        for input_ids, attention_mask, labels in dataloader:
            outputs = model(
                input_ids=input_ids, attention_mask=attention_mask, labels=labels
            )
            logits = outputs.logits
            loss = outputs.loss

            logits = logits.detach().cpu().numpy()
            label_ids = labels.to("cpu").numpy()
            rouge = calc_rouge(logits, label_ids)

            val_loss += loss
            val_rouge += rouge

        val_loss = val_loss / len(dataloader)
        val_rouge = val_rouge / len(dataloader)
        return val_loss, val_rouge

rouge_score = evaluate.load("rouge", tokenizer=tokenizer)
```

- Labels: 정답 요약문으로, 레이블이 -100인 값들을 패딩 토큰의 인덱스로 변경
- Batch_decode: 특수 토큰을 제외하고 토큰 인덱스를 실제 텍스트로 변환. 이 값을 이용해 rouge_score 인스턴스의 compute 메서드로 루지

```
... Downloading builder script: 6.14k/? [00:00<00:00, 151kB/s]
Epoch 1: Train Loss: 2.1932 Val Loss: 1.8763 Val Rouge 0.2359
Saved the model weights
Epoch 2: Train Loss: 1.6207 Val Loss: 1.8891 Val Rouge 0.2520
Epoch 3: Train Loss: 1.2527 Val Loss: 1.9453 Val Rouge 0.2268
Epoch 4: Train Loss: 0.9685 Val Loss: 2.0856 Val Rouge 0.2469
Epoch 5: Train Loss: 0.7413 Val Loss: 2.2326 Val Rouge 0.2322
```

모델 실습

BART 모델 평가

```
model = BartForConditionalGeneration.from_pretrained(  
    pretrained_model_name_or_path="facebook/bart-base"  
) .to(device)  
model.load_state_dict(torch.load("../models/BartForConditionalGeneration.pt"))  
  
test_loss, test_rouge_score = evaluation(model, test_data_loader)  
print(f"Test Loss : {test_loss:.4f}")  
print(f"Test ROUGE-2 Score : {test_rouge_score:.4f}")
```

Loading weights: 100%  259/259 [00:00<00:00, 707.22it/s, Materializing param=model.shared.weight]

Test Loss : 1.8355

Test ROUGE-2 Score : 0.2445

- ROUGE-2는 0에서 1 사이 값을 가지며, 1에 가까울수록 성능 높음

모델 실습

문장 요약문 비교

```
model.eval()

for index in range(5):
    news_text = test.text.iloc[index]
    summarization = test.prediction.iloc[index]

    inputs = tokenizer(
        news_text,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(device)

    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_length=54,
            min_length=10,
            do_sample=False
        )

    predicted_summarization = tokenizer.decode(output_ids[0], skip_special_tokens=True)
    print(f"정답 요약문 : {summarization}")
    print(f"모델 요약문 : {predicted_summarization}\n")
```

```
정답 요약문 : Clinton leads Trump by 4 points in Washington Post: ABC News poll
모델 요약문 : Clinton leads Trump by 4 points: Washington Post-ABC News poll

정답 요약문 : Democrats question independence of Trump Supreme Court nominee
모델 요약문 : U.S. senators question Gorsuch's independence

정답 요약문 : In push for Yemen aid, U.S. warned Saudis of threats in Congress
모델 요약문 : U.S. warns Saudi Arabia over humanitarian situation in Yemen

정답 요약문 : Romanian ruling party leader investigated over 'criminal group'
모델 요약문 : Romanian prosecutors charge party leader with 'criminal group'

정답 요약문 : Billionaire environmental activist Tom Steyer endorses Clinton
모델 요약문 : Environmental activist Steyer backs Clinton for U.S. president
```

ELECTRA



모델 설명

ELECTRA 개요 & 사전 학습 방법

- BERT 등의 자연어 처리 모델은 MLM 방식을 사용하거나 일부 토큰을 MASK로 대체해 입력을 손상시키고 원래 토큰을 복원하는 모델을 학습함
- ELECTRA : 입력을 마스킹하는 대신 생성자(generator)와 판별자(discriminator)를 사용해 사전학습을 수행 (GAN과 유사한 방법으로 학습이 수행)

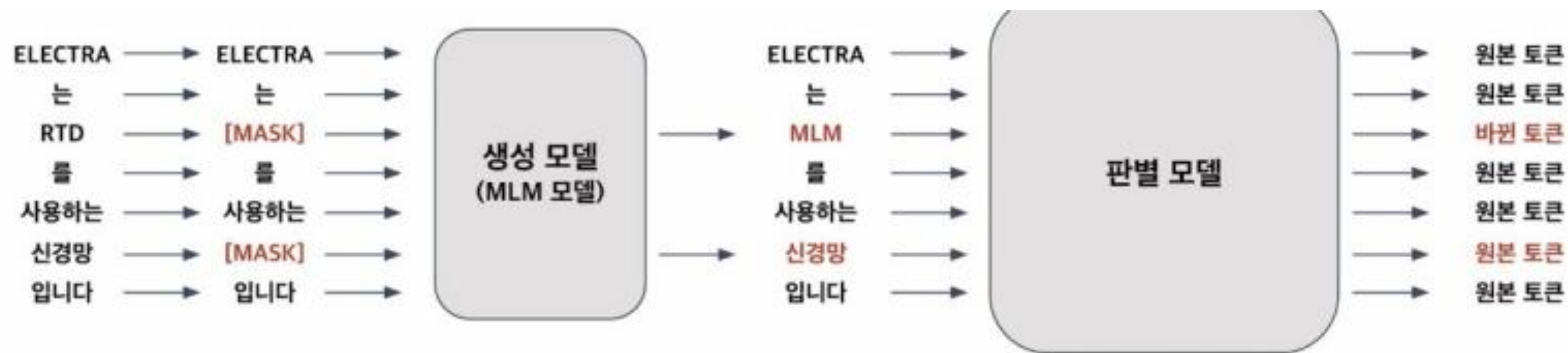


그림 7.20 ELECTRA 모델 학습 방법

모델 설명

사전 학습 방법

- 생성자 모델은 BERT의 마스킹된 언어모델과 동일
- 입력문장의 15%를 마스크 처리
- GAN과의 차이

항목	GAN	ELECTRA
목적	진짜 같은 데이터 생성	언어 표현(representation) 학습
최종 사용 모델	Generator	Discriminator
학습 방식	두 모델 적대적으로 연결 (gradient 공유)	독립적 학습 (MLM + 이진 분류)
Generator 입력	완전한 노이즈 벡터	마스킹된 텍스트
Generator가 정답 맞춘 경우	생성된 것으로 간주 → 가짜 취급	원본 토큰으로 간주 → 진짜 취급
학습 신호 범위	전체 샘플 단위	모든 토큰 단위 (효율적)
가중치 공유	해당 없음	임베딩 계층만 공유, Generator는 Discriminator의 1/2~1/4 크기
다운스트림 활용	Generator 사용	Discriminator만 사용, BERT와 동일하게 fine-tuning

모델 실습

ELECTRA 전처리

- Tokenizer = ElectraTokenizer.from_pretrained

```
7 def make_dataset(data, tokenizer, device):
8     tokenized = tokenizer(
9         text=data.text.tolist(),
10        padding="longest",
11        truncation=True,
12        return_tensors="pt"
13    )
14    input_ids = tokenized["input_ids"].to(device)
15    attention_mask = tokenized["attention_mask"].to(device)
16    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
17    return TensorDataset(input_ids, attention_mask, labels)
18
19
20 def get_dataloader(dataset, sampler, batch_size):
21     data_sampler = sampler(dataset)
22     dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
23     return dataloader
24
25
26 epochs = 5
27 batch_size = 32
28 device = "cuda" if torch.cuda.is_available() else "cpu"
29 tokenizer = ElectraTokenizer.from_pretrained(
30     pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
31     do_lower_case=False,
32 )
```


모델 실습

ELECTRA 모델구조

- ElectraForSequenceClassification.from_pretrained
- BERT 와 마찬가지로 12개의 인코더 계층을 통과해 입력
- 텍스트의 상태값을 반환
- 분류기 계층은 CLS 토큰의 벡터를 이용해 입력 텍스트에
- 대한 분류를 수행

[4]



```
1 from torch import optim
2 from transformers import ElectraForSequenceClassification
3
4
5 model = ElectraForSequenceClassification.from_pretrained(
6     pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
7     num_labels=2
8 ).to(device)
9 optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

```
... electra
    └─ embeddings
        │   └─ word_embeddings
        │   └─ position_embeddings
        │   └─ token_type_embeddings
        │   └─ LayerNorm
        │   └─ dropout
    └─ encoder
        │   └─ layer
        │       │   └─ 0
        │       │   └─ 1
        │       │   └─ 2
        │       │   └─ 3
        │       │   └─ 4
        │       │   └─ 5
        │       │   └─ 6
        │       │   └─ 7
        │       │   └─ 8
        │       │   └─ 9
        │       │   └─ 10
        │       │   └─ 11
    └─ classifier
        │   └─ dense
        │   └─ activation
        │   └─ dropout
        │   └─ out_proj
```

모델 실습

ELECTRA 평가

- BERT(80.85%)에 비해 7.5% 높음
- GLUE(문장 수준 또는 문서 수준의 이해력을 평가하는 데이터세트)에서 우수한 성능을 보임
- 동일한 구조와 가중치 개수를 가지지만 BERT보다 우수하다

KoELECTRA모델 학습 결과

```
Epoch 1: Train Loss: 0.4489 Val Loss: 0.3227 Val Accuracy 0.8715
Saved the model weights
Epoch 2: Train Loss: 0.2891 Val Loss: 0.2928 Val Accuracy 0.8808
Saved the model weights
Epoch 3: Train Loss: 0.2188 Val Loss: 0.3111 Val Accuracy 0.8802
```

KoELECTRA모델 평가 결과

```
Test Loss : 0.3084
Test Accuracy : 0.8732
```

T5



모델 개요

T5 개요

- 2019년 구글에서 발표 – 트랜스포머 구조를 바탕으로 함
- 인코더 – 디코더 구조를 바탕으로 여러 평가에서 탁월한 성능을 보임
- 기존의 자연어 처리 모델 : 입력 문장을 행렬/벡터로 표현한 뒤 출력 문장을 생성/출력값이 클래스 또는 입력값의 일부를 반환
- T5 : 입출력을 모두 토큰 시퀀스로 처리 -> 유연성과 확장성이 뛰어나다
- Ex) 문장번역, 요약, 질의 응답, 텍스트 분류

모델 개요

T5 개요

- 입출력이 모두 토큰이므로 미세한 조정이 필요 (사전학습 후 미세조정 단계에서 해당 작업의 데이터를 활용)
- CoLA : 문법적으로 허용
- 가능한 문장 판별
- STS-B : 문장 간 유사성 측정

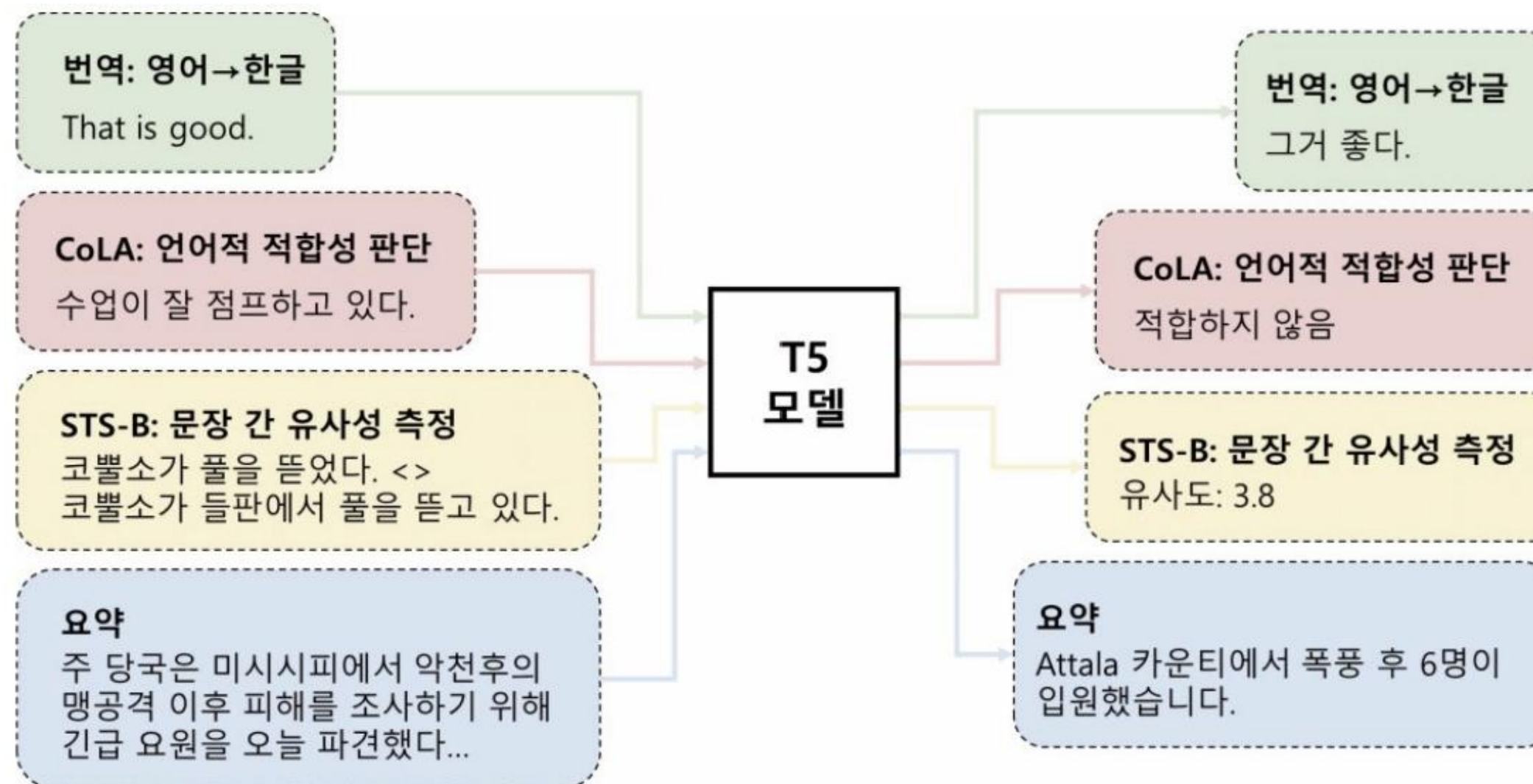


그림 7.21 T5 모델 학습 유형

모델 개요

T5 사전학습방법

- 입력 문장의 일부 구간을 마스킹해 입력 시퀀스 처리
- 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결
-
- 문장마다 유일한 마스킹 토큰이라는 뜻의 센티넬 토큰 사용
- T5는 마스킹 토큰을 예측하는 것을 목적으로 함

모델 개요

T5 미세 조정 방법

- 지도학습
- 문장이 인코딩되기 전에 'translate to German', 'summarize'와 같은 작업 토큰을 추가한다

모델 실습

T5 전처리

- BART와 달라진 점 : max_length = 128, pad_to_max_length = True, return_tensors = 'pt'
- Pad_to_max_length : 최대길이보다 짧으면 패딩, 길면 문장 자름
- 토큰인덱스(input_ids), 어텐션 마스크 반환, 토큰화 결과 출력

```
8 def make_dataset(data, tokenizer, device):
9     source = tokenizer(
10         text=data.text.tolist(),
11         padding="max_length",
12         max_length=128,
13         pad_to_max_length=True,
14         truncation=True,
15         return_tensors="pt"
16     )
17
18     target = tokenizer(
19         text=data.prediction.tolist(),
20         padding="max_length",
21         max_length=128,
22         pad_to_max_length=True,
23         truncation=True,
24         return_tensors="pt"
25     )
26
27     source_ids = source["input_ids"].squeeze().to(device)
28     source_mask = source["attention_mask"].squeeze().to(device)
29     target_ids = target["input_ids"].squeeze().to(device)
30     target_mask = target["attention_mask"].squeeze().to(device)
31     return TensorDataset(source_ids, source_mask, target_ids, target_mask)
32
33 def get_dataloader(dataset, sampler, batch_size):
34     data_sampler = sampler(dataset)
35     dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
36     return dataloader
```

- T5Tokenizer.from_pretrained

```
39 epochs = 5
40 batch_size = 8
41 device = "cuda" if torch.cuda.is_available() else "cpu"
42 tokenizer = T5Tokenizer.from_pretrained(
43     pretrained_model_name_or_path="t5-small"
44 )
```

모델 실습

T5 모델학습

- T5의 shared 임베딩 : 인코드&디코더에 사용되는 토큰 임베딩 함수, 공유되어 사용됨
- T5모델은 토큰 인덱스, 어텐션 마스크, 디코더 토큰 인덱스, 라벨로 모델을 학습함
- Decoder_input_ids : 디코더에 입력될 시퀀스를 입력한 텐서, 마지막을 제외한 나머지 토큰 사용
- Label : 디코더인풋인덱스보다 다음 시점 예측하도록 target_ids[:,1:]로 설정

```
def train(model, optimizer, dataloader):  
    model.train()  
    train_loss = 0.0  
  
    for source_ids, source_mask, target_ids, target_mask in dataloader:  
        decoder_input_ids = target_ids[:, :-1].contiguous()  
        labels = target_ids[:, 1:].clone().detach()  
        labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100  
  
        outputs = model(  
            input_ids=source_ids,  
            attention_mask=source_mask,  
            decoder_input_ids=decoder_input_ids,  
            labels=labels,  
        )
```

모델 실습

T5 모델학습

- Pad_token_id = -100 : 손실값 무시
- Contiguous : 텐서를 메모리에 연속된 블록으로
저장

```
def train(model, optimizer, dataloader):  
    model.train()  
    train_loss = 0.0  
  
    for source_ids, source_mask, target_ids, target_mask in dataloader:  
        decoder_input_ids = target_ids[:, :-1].contiguous()  
        labels = target_ids[:, 1:].clone().detach()  
        labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100  
  
        outputs = model(  
            input_ids=source_ids,  
            attention_mask=source_mask,  
            decoder_input_ids=decoder_input_ids,  
            labels=labels,  
        )
```

모델 실습

T5 모델 테스트

빔 서치 동작 요약

매 스텝마다 후보 시퀀스를 num_beams개만 유지
→ 나머지 삭제 → 반복 → 최종적으로 확률 가장
높은 시퀀스 출력

한계

T5 요약 결과가 실제 요약과 **내용은 유사하나**
완전히 일치하진 않음 → 더 많은 학습 데이터 +
하이퍼파라미터 튜닝으로 개선 가능

파라미터	역할	값 효과
<code>input_ids</code>	입력 문장 인코딩	-
<code>attention_mask</code>	마스킹 정보	-
<code>max_length</code>	출력 시퀀스 최대 길이	-
<code>num_beams</code>	빔 서치 빔 크기	클수록 후보 많이 유지 → 품질 ↑, 속도 ↓
<code>repetition_penalty</code>	중복 토큰 억제	높을수록 반복 억제 ↑
<code>length_penalty</code>	시퀀스 길이 보상	높을수록 긴 출력 생성
<code>early_stopping</code>	EOS 토큰 생성 시 조기 종료	<code>True</code> 시 <code>max_length</code> 전에도 중단